A SOURCE-BASED REVIEW

Further Lean proof simplifications

Pell proof reductions and canonical residue witnesses

Prepared for Vladimir Reshetnikov
26 October 2026

Scope. ProveIt / Computability / DiophantineConsistency. Earlier proposals in the requested research and incoming directories are treated as already applied, including optional or uncompiled proposals.

Source baseline. https://github.com/VladimirReshetnikov/ProveIt/commit/96520cd1a5179f470b4da127bd47bc26a8d808ff. A closing comparison is documented separately.

Delivery. This manuscript, its compiled PDF, Lean prototypes, exact source patches, novelty ledgers, validation receipts, and replay instructions.

Contents

1 Results and recommendations 2
1.1 What makes the proposals useful . . . 2

2 Scope and the novelty standard 3
2.1 What was counted as already available . . . 3
2.2 Nearest prior proposals and the remaining obligation . . . 3

3 Two small Pell proof reductions 4
3.1 Feed the square identity directly to Pell classification . . . 4
3.2 Use the integer Pell norm theorem where the goal is integer-valued . . . 5
3.3 Expected effect and limits . . . 5

4 Decode bounded witnesses by canonical residues 6
4.1 The unnecessary choice . . . 6
4.2 The precise arithmetic contract . . . 6
4.3 The client becomes deterministic after choosing the row prime . . . 7
4.4 Boundary cases and why primality is essential . . . 7
4.5 Placement and cost . . . 7

5 A larger candidate withdrawn after the closing review 8

6 Verification, reproducibility, and limits 9
6.1 Two snapshots, one unchanged set of target files . . . 9
6.2 Exactly what the Lean checks establish . . . 9
6.3 Source accounting and what was not measured . . . 10
6.4 Reproduction . . . 10

7 Adoption order and review obligations 10
7.1 What to measure after integration . . . 11

8 Source and artifact guide 11
8.1 Contents of the delivery . . . 12

1 Results and recommendations

Three recommendations survive the final novelty screen. Two are small, immediately useful Pell proof reductions. Together they remove ten physical lines, without introducing helpers or imports. The third replaces arbitrary bounded-witness selection with a canonical residue function; it simplifies the proof contract but adds eleven lines across its shared helper and two clients.

The largest initially promising candidate was withdrawn after a closing review found a newly added incoming proposal that already supplies its architectural payoff. It is retained only as screened-out evidence. This is intentional: the requested assumption that all earlier proposals are already applied takes priority over producing a longer list.

Recommendation	Exact change	Source accounting	Validation scope
1. Integer Pell norm	Use `pell_eqz/dz_val` for two integer norm goals in `ThirteenFreshSeed`.	217 to 210 lines; 7 removed; no helper/import cost.	Complete modified module passed with cached dependencies.
2. Direct Pell-entry adapter	Use `Nat.sub_eq_of_eq_add'` on the existing square identity in `SignedPellResidue`.	102 to 99 lines; 3 removed; no helper/import cost.	Complete modified module passed with cached dependencies.
3. Canonical residue witness	Prove the prime descending-product criterion; set `j i := a i mod p`.	Shared helper +17; two clients each -3; net +11 lines.	Arithmetic helper passed; focused decoder and actual-module scope are detailed in Section 6.

All three retain the existing public theorem statements and certificate data. No source-size saving is claimed by adding the negative and positive counts across unrelated optional changes. The eleven-line cost of the canonical interface remains a cost even though the two Pell changes save ten lines elsewhere.

1.1 What makes the proposals useful

The Pell changes consume facts in the form already available: an additive natural square identity on entry to classification, and an integer norm invariant for integer-valued goals. The residue change gives the decoder a named arithmetic contract with a concrete witness. These eliminate identifiable proof obligations, rather than just making tactic scripts more opaque.

The report supplies exact replacements, mathematical justifications, the closest earlier proposals, counterexamples to an overgeneralized statement, measured source costs, and explicit verification limits. Read Section 3 for the two low-cost edits, Section 4 for the canonical decoder, and Section 6 before applying any patch. The repository was inspected and candidate copies were checked; no upstream files or branches were changed.

2 Scope and the novelty standard

The review uses the source at commit `96520cd1a5179f470b4da127bd47bc26a8d808ff`. The main source directory contains 541 direct Lean modules. The requested research directory contains 43 dated proposal packets, a through z and za through zq, at that baseline. The incoming directory also contains a substantive ZIP in addition to its routing README. Its five proposals were extracted and included in the exclusion set. The closing comparison described in the validation section extends this screen to the later inspected head.

The language version is pinned by the repository to Lean 4.32.0. Its manifest pins Mathlib to `81a5d257c8e410db227a6665ed08f64fea08e997`. These pins matter: theorem names, simplifier behavior, and import costs are properties of this environment, not generic assurances about any Lean installation. The source manifest and validation receipts in the archive make the inspected and checked files identifiable.

2.1 What was counted as already available

The exclusion rule is deliberately stronger than "not yet merged." A previous optional interface, incomplete prototype, rejected-but-already-described direction, or a mechanism proposed for a nearby client is prior work for this task. Renaming a helper, adding a parameter, moving a tactic proof into a wrapper, or applying the same abstraction to another file does not establish a fresh idea.

Three parallel audits reviewed the primary manuscripts and substantive notes for packets a–m, n–z, and za–zq plus incoming. The archive retains those ledgers. Searches were used to locate close comparisons, followed by review of the relevant proposal and source context; an absent identifier alone was not treated as proof of novelty. The historical incoming README also discusses unrelated equation and K215 research. The audit covers its relevant Lean simplification content and the tracked incoming archive, not every manuscript transitively linked from that historical queue.

Several initially attractive directions were discarded on this basis: generic finite-sum rigidity; additional carry uniqueness or reset locality; scanner fusion; source-register decoding abstractions; aligned-word equivalences; shared Pell growth/error estimates; finite-controller synthesis; native coefficient envelopes; stronger arbitrary-subset coverage; and standard import/profiling advice. They would repeat earlier work.

2.2 Nearest prior proposals and the remaining obligation

Candidate	Already assumed available	Distinct remaining change
Pell input equation	Whole-conclusion casts, signed congruence composition, and odd-index reuse (packet k, packet m).	Consume the existing natural square identity at entry to `Pell.eq_pell`; the old subtraction bridge survives the prior edits.
Integer Pell norm	Integer-only norm reasoning, fundamental-unit criteria, and packet ze's fresh-seed changes.	Reuse `pell_eqz` for two constructed integer norms. Packet ze's source retains both cast bridges.

Candidate	Already assumed available	Distinct remaining change
Canonical residue	CRT congruence cleanup in packet n; canonical division and pairing witnesses elsewhere.	The exact prime-product interval criterion yields `a i % p`. Packet n leaves the falling-factorial branch unchanged.

The screened-out support-bound candidate is discussed in Section 5; its advertised simplification is already supplied by the closing incoming conservation proposal.

These distinctions support narrow claims about the supplied exclusion corpus. They are not claims of global mathematical originality. In particular, geometric series, least nonnegative residues, and direct reuse of a library invariant are familiar mathematics. The contribution here is identifying the exact unnecessary obligations in these proofs after the earlier suggestions are assumed present.

3 Two small Pell proof reductions

These are the recommended first changes. Both preserve theorem statements and use already imported infrastructure. Neither adds a helper, an import, a stronger hypothesis, or new executable data.

3.1 Feed the square identity directly to Pell classification

The target is `SignedPellResidue.signed_residue_cases`, especially the block beginning at baseline line 62. The preceding argument already proves

$$(A\,|V|)^2 = (A^2-1)|y|^2 + 1$$

as an equality of natural numbers. The current proof then creates another equality with explicit multiplication, calls `nlinarith`, uses `omega` to turn addition into truncated subtraction, and finally passes that fact to `Pell.eq_pell`.

The additive equality is exactly the premise of `Nat.sub_eq_of_eq_add'`. It already contains the information that makes the natural subtraction exact. Normalize squares and multiplication association at the point of use:

```
obtain ⟨n, hx, hy⟩ := Pell.eq_pell hA
    (x := A * V.natAbs) (y := y.natAbs) (by
  change (A * V.natAbs) * (A * V.natAbs) -
    (A * A - 1) * y.natAbs * y.natAbs = 1
  simpa only [pow_two, mul_assoc] using
    Nat.sub_eq_of_eq_add' hn)
```

The apostrophe is significant: it selects the orientation of the addition equation used here. The explicit `change` exposes the discriminant hidden behind the imported `Pell.d` definition; restricted simplification alone does not unfold that interface. Both details are present in the successfully checked patch.

The change removes the local `hp` bridge and three physical source lines: the complete module goes from 102 to 99 lines. It leaves the absolute-value conversion that produces `hn`, the index-parity argument, and all subsequent sign/congruence cases intact. Those later stages are adjacent to earlier proposals and receive no new credit here.

A possible alternative was to import an existing `eq_pell_of_sq` wrapper from a heavier module. That route is not recommended: this short adapter already exposes the exact mathematical step, while the heavier dependency would outweigh the small convenience. The supplied patch adds no dependency.

3.2 Use the integer Pell norm theorem where the goal is integer-valued

The target is `ThirteenFreshSeed.auxiliary_exists`. Two local goals assert an integer norm identity for Pell sequence values. The baseline starts from `χ_sq`, an identity over natural numbers, casts it with `congrArg`, proves or supplies a guard for natural subtraction, and normalizes the resulting integer expression.

Pinned Mathlib already provides the integer invariant `Pell.pell_eqz`. Its discriminant conversion is `Pell.dz_val`. These are available through the module's existing imports. For the first norm identity, the entire proof becomes:

```
have hp : f ^ 2 - delta * (yn hA t : ℤ) ^ 2 = 1 := by
  simpa only [f, delta, xz, yz, dz_val, az,
    pow_two, mul_assoc] using pell_eqz hA t
```

For the second, use the same integer invariant, substitute the existing coordinate equation `hx`, and retain the short polynomial normalization needed for the chosen sign of `y`:

```
have hnorm : (S : ℤ) ^ 2 * V ^ 2 -
    ((S : ℤ) ^ 2 - 1) * y ^ 2 = 1 := by
  have h := pell_eqz hS R
  simp only [xz, yz, dz_val, az, hx] at h
  dsimp [y]
  nlinarith only [h]
```

This removes four lines at the first site and three at the second. The complete module goes from 217 to 210 physical lines. The local `hSS` subtraction guard disappears because the direct integer theorem already packages the correct discriminant. Other guards and casts used elsewhere in the construction remain necessary and are not removed.

The useful principle here is specific: choose the invariant in the scalar domain of the target equation. It is not a new general casting framework. Earlier integer-norm proposals address a different obligation, such as avoiding rational square arguments or minimizing a fundamental unit. This change substitutes an existing theorem for two constructive sequence-norm bridges. The earlier fresh-seed replacement itself retains those bridges, which provides a particularly close counterfactual comparison.

3.3 Expected effect and limits

Taken together, these two patches remove ten physical lines with no helper overhead. They reduce intermediate facts and two uses of natural-subtraction cast machinery. No compile-time, memory, or object-size improvement has been established. The validation section distinguishes complete modified-module checks from a project rebuild, and records the dependencies used. Their small scope makes these the best initial adoption candidates even though their absolute line savings are modest.

4 Decode bounded witnesses by canonical residues

The target is the forward direction of `BoundedForallCertificate.boundedForallCertificate`, baseline lines 32–63. Its arithmetic argument is also present in the lower-level `MRDP.boundedForall_dioph`, in the sibling HilbertTenthProblem source tree. That second site matters for the placement and cost of a shared helper; it is not an additional independent proposal.

4.1 The unnecessary choice

The certificate contains, for every witness coordinate i, a divisibility condition

$$P \mid \prod_{t=0}^{u_i} (a_i - t),$$

where subtraction and the product are over natural numbers. After choosing a prime p dividing the row modulus, the proof knows that p divides this descending product. It currently selects an arbitrary factor divisible by p, obtains an index t, and uses $\min(a_i, t)$ to repair the effect of truncated subtraction. A `choose` then assembles these choices into a witness function.

The downstream proof needs just two properties of this function: its coordinates are bounded by u, and each coordinate is congruent to the corresponding a_i modulo p. There is a distinguished value with the congruence property: the least nonnegative residue a_i mod p. The product condition says exactly that this distinguished value lies inside the required interval.

4.2 The precise arithmetic contract

For a prime p and arbitrary natural numbers a, u, prove

$$p \mid \prod_{t=0}^{u} (a - t) \quad \iff \quad a \bmod p \le u. \tag{1}$$

There is no assumption $u < p$, no positivity condition on a, and no conversion of natural subtraction to integer subtraction in the theorem statement. The supplied kernel-checked theorem is:

```
theorem prime_dvd_descendingProduct_iff {p a u : ℕ}
    (hp : p.Prime) :
    p ∣ ∏ t ∈ Finset.range (u + 1), (a - t) ↔
      a % p ≤ u
```

For the forward direction, primality gives some $t \le u$ with $p \mid a - t$. Set $b = \min(a, t)$. Natural subtraction gives $a - t = a - b$, and $b \le a$ permits the congruence step. Therefore

$$a \bmod p = b \bmod p \le b \le t \le u.$$

The minimum is used only inside this small arithmetic lemma. It disappears from the certificate decoder and its chosen witness.

For the reverse direction, let $r = a \bmod p \le u$. The factor at index r occurs in the product, and the standard remainder identity gives $p \mid a - r$. Divisibility of the product follows immediately. This also explains why the canonical residue is the right witness: it identifies a specific factor when the interval test holds.

4.3 The client becomes deterministic after choosing the row prime

The replacement block is:

```
let j := fun i => a i % p
have hWitness_bound i : j i ≤ u i :=
  (prime_dvd_descendingProduct_iff hp).mp
    (hPrime_product.trans (hWitness_product i))
have hWitness_congruence i : a i ≡ j i [MOD p] :=
  (mod_modEq _ _).symm
```

The rest of the argument keeps its structure. Monotonicity of the nonnegative polynomial evaluations supplies the row bound. Coprimality with the factorial scale puts that bound below p. Cancelling the scale recovers the row-index congruence, polynomial congruence transports evaluations, and the below-p bounds turn congruence into equality. The supplied focused decoder check retains these downstream steps so that the replacement is tested beyond the standalone number-theoretic fact.

The phrase "deterministic decoder" has a limited meaning here: once the row prime p has been selected, j is a concrete residue function. The proof still chooses a prime divisor of the row modulus. It still uses the existing CRT construction in the reverse direction. This is not a claim that the whole certificate development becomes constructive, choice-free, or computationally cheaper.

The polynomial certificate, quantifier arity, number of witnesses, products, and stored syntax stay unchanged. In particular, the proposal does not replace the falling-factorial condition with a modulo expression in the Diophantine formula. The modulo operation appears only in the mathematical decoding proof. This distinction preserves the representation and depth/cost obligations that other parts of the development depend on.

4.4 Boundary cases and why primality is essential

For $u = 0$, the product has one factor a, and the equivalence reduces to $p \mid a$ if and only if $a \bmod p = 0$. If $a \leq u$, the product contains a zero factor, and the residue bound holds because $a \bmod p \leq a$. If $u \geq p - 1$, every least residue lies in the interval, exactly as the statement predicts. No large-prime side condition is being smuggled into the helper.

Primality cannot be replaced by mere nonzeroness. For $p = 6$, $a = 3$, and $u = 1$, the product is $3 \cdot 2 = 6$, so p divides it, but $a \bmod p = 3 > 1$. This small counterexample catches the tempting but invalid composite-modulus version. The prime-selection step in the original decoder is therefore retained for a mathematical reason.

The finite coordinate type can be empty: a residue function on `Fin 0` is well-defined and its coordinate obligations are vacuous. If the outer row range is empty, the original bounded-universal statement already handles that case. The change adds no special-case branch.

4.5 Placement and cost

A shared helper belongs at the lower arithmetic layer already imported by the certificate theorem, or in a small dependency imported by that layer. Placing it only above `BoundedForallCertificate` would prevent the lower `MRDP.boundedForall_dioph` copy from using it without reversing the dependency direction. The candidate therefore includes the lower-layer edit and the requested target edit together.

This is a simplification of witness selection and proof interfaces, not an immediate net source reduction. In the supplied formatting, the helper costs 17 lines, and each of the two client blocks shrinks by three lines, for eleven extra lines overall. A single-client adoption would cost fourteen lines. The complete iff is useful as a clean arithmetic contract, but only its forward implication is required by the two clients. If minimizing source lines is the sole objective, these overheads argue for postponing the helper until its interface value or another client justifies it.

The stronger reason to adopt is stability: every later reference to the decoded coordinate has a definitional description, rather than depending on an arbitrary factor witness and an index correction. Earlier CRT simplifications in packet n address congruence composition and explicitly leave this falling-factorial branch in place. Earlier canonical pairing or division witnesses do not establish equivalence (1). The remaining obligation after those proposals is therefore identifiable and different.

5 A larger candidate withdrawn after the closing review

An initially promising candidate targeted `CurrentFiniteDummyMutation2011.exists_normalization_sum`: establish a bound for the entire set of safe dummy updates, then normalize the final Boolean word once instead of constructing an existential normalization at every finite-set insertion. A standalone quantitative theorem and an endpoint transport theorem were successfully checked.

Write $M = 2011N$, let $\rho \geq 2$, and suppose $K < \rho^{2022}$. For a finite set S of cells satisfying the actual safety inequalities

$$j.\mathrm{val} + 1 < N, \qquad j.\mathrm{val} + s < N,$$

the checked estimate is

$$(K + \rho^{2011s}) \left(\rho^{977} \sum_{j \in S} (\rho^{2011})^{j.\mathrm{val}} \right) < \rho^{M-2}.$$

It follows by bounding the kernel and temporal parts using injective exponent maps and a geometric sum. It needs no cardinality limit on S and includes $N = 1$, when safety forces S to be empty. The numerical statement is sound, and distinct from the earlier point-update equality by itself.

However, a new incoming review added during this task proposes exact boundary-carry conservation. Under the user's required counterfactual baseline, that conservation contract is already available. Its route can remove the same normalization/nonwrap bookkeeping without introducing the aggregate numerical bound. Thus the initially advertised architectural benefit is not a sound basis for another simplification recommendation.

Decision: withdraw this candidate from the recommendation set. The fact that its numerical theorem is different does not suffice when its proposed simplification payoff is already supplied by the assumed baseline. It is also not a source-compression win as delivered: the client adapter removes 25 lines but the standalone helper costs 122, yielding 97 extra lines if copied literally. Its actual project adapter was not compiled.

The archive retains this work only in `screened_out/`, together with the late novelty decision. It is excluded from every count of recommended changes and savings. The two-window capacity lemma and ordinary geometric identities reproduced inside that prototype are explicitly baseline infrastructure, not new proposals. This record is included to make the novelty screen auditable, rather than silently relabeling a superseded idea.

6 Verification, reproducibility, and limits

6.1 Two snapshots, one unchanged set of target files

The main audit and patches use `96520cd1a5179f470b4da127bd47bc26a8d808ff`. A closing comparison inspected `3b51c69d42ff99bc420698532af3fe1d1f9bd356`, committed at 19:32:32 UTC on 26 October 2026. The interval contains thirteen commits and thirteen changed paths, including five maintained Lean files and six new incoming ZIPs. None of the three recommended target modules changed; their separately fetched closing blobs match the original local bytes.

The six new archives were integrity-checked and their substantive proposal sections reviewed. Their exact Git blob hashes, SHA-256 hashes, manuscript members, and the resulting novelty decisions appear in `evidence/late-head-audit.json` and its readable companion. The most consequential result was the withdrawal in Section 5. This is a frozen closing audit, not a claim about commits added after that snapshot.

Unchanged target bytes support applicability of the local edits at those locations. They do not establish a full build against all changed dependencies. Nor do they establish a build of the hypothetical combined tree in which every prior suggestion has been implemented; that tree was not supplied. The earlier proposals are used as the semantic exclusion baseline, while actual checking is against the identified source and available dependency objects.

6.2 Exactly what the Lean checks establish

The compiler is Lean 4.32.0, release commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`. Its available dependency cache follows the repository's pinned Mathlib revision. The direct project import sources used by the Pell checks were compared with the pinned Git tree; all matched. This is explicitly a cached-dependency validation, not a clean rebuild of the full transitive dependency graph.

Artifact	Result	Scope
`ThirteenFreshSeedAudit.lean`	Pass	Complete modified current module, followed by axiom queries for `auxiliary_exists` and `fresh_private_seed`.
`SignedPellResidueAudit.lean`	Pass	Complete modified current module, followed by axiom queries for `signed_residue_cases` and `four_square_dvd_sub_one`.
`CanonicalResidue.lean`	Pass	Standalone prime descending-product iff, with no project imports.
`CanonicalDecoder.lean`	Pass	Focused forward decoder, including factorial bounds and polynomial congruence; the existing encoded-moduli product identity is an explicit input.

Actual-module integration remains unverified. The complete lower `MRDPCore` module stopped at a missing cached `Mathlib.NumberTheory.Dioph` import. Both real-module canonical sources are integration candidates, not full-module passes. The withdrawn aggregate result was checked only as a standalone kernel; its project adapter was not compiled.

The focused decoder abstracts only the existing encoded-moduli product equality, exposed as a premise. It retains the witness bounds, factorial separation, row-index cancellation, polynomial evaluation congruence, and final equality of bounded evaluations. It does not check the reverse CRT construction, the outer Diophantine syntax theorem, or the complete actual-module import graph. An unchanged small Mathlib prime/factorial source needed for this focused check was compiled locally; its provenance and receipt are included.

The successful inspected endpoint axiom sets are `propext`, `Classical.choice`, and `Quot.sound`. No `sorryAx` is accepted in a successful final receipt. This records absence of an admitted proof at those checked endpoints; it is not a stronger foundational or dependency-minimality claim. Exact checked source hashes and complete compiler output are included, so a source file cannot be silently substituted for the one actually checked.

6.3 Source accounting and what was not measured

All physical line counts include comments and blank lines in the exact files. Displayed code excerpts may be wrapped for page layout; measurements count the delivered source files. The Pell patches add no helper or import and save ten lines altogether. For the canonical decoder, `MRDPCore` changes from 391 to 405 lines, while `BoundedForallCertificate` changes from 98 to 95: the combined change is +11. These are measurements of the literal pinned source, not invented counts for a hypothetical merged tree after all prior proposals.

No timing comparison with the unmodified modules was run. Recorded compiler durations are execution receipts, not performance benchmarks. No claim is made about memory, object size, generated equation size, or runtime speed. No complete project build or downstream release gate was run. These limits are material to integration but do not diminish the exact small-module and focused arithmetic evidence reported above.

6.4 Reproduction

Use a checkout with the stated Lean and Mathlib pins and build its normal dependencies. The archive's `scripts/replay_checks.sh` invokes the four recommended checked sources through `lake env lean`; it does not apply patches or alter the checkout's source. Actual integration requires applying the relevant patch and running the repository's normal module/check targets. The canonical pair still needs that real-module integration check.

Run `scripts/build_report.sh` from an environment with XeLaTeX and the documented fonts to regenerate the PDF. The archive includes `SHA256SUMS` for every delivered file other than the checksum file itself. The PDF was rendered and inspected after compilation; the source is included so the report remains editable.

7 Adoption order and review obligations

First: the integer norm replacement. Apply `ThirteenFreshSeed.patch` as an isolated proof edit. It has the clearest dependency story: both replacement theorems are already imported, the exact goals are integer-valued, and the two old cast bridges are visibly redundant. Compile the module and its ordinary downstream checks in the intended checkout. The supplied complete-module check is good local evidence, but does not substitute for the repository's own integration gates.

Second: the direct Pell-entry adapter. Apply `SignedPellResidue.patch` separately. Keep the explicit change of target needed to expose the discriminant before the restricted simplification. The small reduction comes from using the additive natural equality already present. Avoid introducing a broad global simp attribute or a heavier wrapper import for this one use.

Third: decide whether a canonical decoding contract is worth its source cost. Review the arithmetic equivalence and the focused decoder first. If the deterministic witness improves maintainability, place the helper at the shared lower layer, apply both clients together, and rebuild their actual imports and checks. Check that the public theorem types and the polynomial/certificate expressions remain byte-for-byte unchanged outside the intended proof bodies. The patch is intentionally a proof-side change; adopting a new modulo-based certificate would be a different project.

Do not apply the screened-out normalization adapter as another recommended simplification. Its mathematical estimate can be retained separately if a future quantitative result requires it, but the newly available conservation interface is already assumed in this task's baseline.

7.1 What to measure after integration

The reported line counts are physical lines in the exact baseline and candidate files, with helpers charged in full. They are useful for checking the narrow compression claim. They do not measure proof-term size, elaboration work, conceptual simplicity, or the downstream cost of an import. No runtime improvement is promised.

For these small edits, normal module compilation and the repository's existing checks are appropriate. A separate benchmark campaign is unnecessary unless elaboration actually regresses. For the canonical helper, review the semantic interface and the two real clients before adding more generic variants. The supplied composite-modulus counterexample is a more useful guard against an incorrect generalization than a collection of tests mirroring the proof text.

The arithmetic source statements, literal large constants, generated programs, certificate dimensions, and theorem hypotheses are not changed by the proposed patches. Preserving them is a concrete part of the design, because changes to executable data or representation cost would invalidate unrelated downstream equalities even if the high-level mathematics stayed true.

8 Source and artifact guide

All repository links below are immutable source references. The archive's evidence directory records exact file hashes and the closing comparison, so these references do not rely on the moving `main` branch.

1. `SignedPellResidue.signed_residue_cases`, original lines 51–74: the natural square identity, temporary subtraction equation, and call to `Pell.eq_pell`.
2. `ThirteenFreshSeed.auxiliary_exists`, original lines 74–150: both constructive integer norm goals. The linked complete file also shows the unchanged public theorem and imports.
3. `BoundedForallCertificate.boundedForallCertificate`, original lines 21–63: the exact certificate type and the forward decoder. Its lower-level duplicate is `Diophantine.Common.MRDPCore`.
4. Pinned Mathlib: `PellMatiyasevic.lean`, especially `xz`, `yz`, `dz_val` at line 167 and `pell_eqz` at line 222.
5. The 43 baseline proposal packets; the archive contains separate early, middle, and latest exclusion ledgers with precise manuscript anchors.
6. The incoming directory at the closing audit pin. The late-head audit identifies each additional archive and the reason for the withdrawn candidate.

8.1 Contents of the delivery

`report.tex` and **`sections/`**
: Complete editable LaTeX source. Build with XeLaTeX; no network access or generated image asset is needed.

`report.pdf`
: Compiled and visually inspected report.

`lean/`
: Checked standalone arithmetic sources, a focused decoder, and complete modified Pell modules. The README states which files are integration candidates rather than checked production modules.

`patches/`
: Exact unified patches against the source baseline, including both canonical-decoder clients and the shared helper placement.

`validation/`
: Compiler output, status receipts, source hashes, and dependency provenance. A failed full MRDP import attempt is identified separately from successful focused checks.

`evidence/`
: Novelty ledgers, independent review, source manifest, and closing-head comparison.

`screened_out/`
: The withdrawn aggregate-support candidate and its explicit non-recommendation notice.

`scripts/`, **`README.md`**, **`SHA256SUMS`**
: Reproduction instructions, report build script, validation replay script, and integrity manifest.